可追溯、可校准的 A 股投研多智能体系统：从年报到深度研报

陈沫樵[1,*,†]　许冰洁[1,*]　赵千淳[2,*]　刘盈羽[1,*]

[1] 对外经济贸易大学　　[2] 中国人民大学

mochiaochen@gmail.com

代码：https://github.com/MochiaoChen/finagent（私有仓库，评审可申请访问）

摘要

大语言模型（LLM）直接“读年报、写研报”有三类在投研场景中不可接受的失败：编造或记错财务数字；在增速、占比与估值上心算出错；用参数记忆补写没有出处的外部信息。此外，研报质量本身缺少可复核的客观评价，使“改进”难以度量。本文介绍 FinAgent，一个面向 A 股上市公司的可追溯投研多智能体系统，覆盖财报分析、估值建模与研报纠错三项任务，并能把一份年报加工为 25 ~ 32 页、60 个以上图表的卖方体例深度研报和公式联动的估值模型。

系统的核心设计是**理解与计算分离**：模型负责读、抄、写与判断，确定性程序负责校验、计算与拦截。具体包括：（1）证据约束的双通道抽取，每个数字带页码级来源并逐字回查原文，规则通道与模型通道冲突时不静默取舍；（2）单一真值表驱动的分析与估值链（分部量价 → 三表联动 → FCFF 折现 → 股权价值桥 → 程序评级）；（3）按展示精度设定容差的**数字护栏**与约束改写的**数字锁**；（4）由五个上下文隔离的无头会话和 11 道程序闸门组成的研报流水线，先搜集外部资料、后分析写作；（5）四层评分标准与盲评闭环，评分标准对生成方不可见，并用重复评审、程序制造缺陷与人工复核三组证据校准评审模型本身。

在三家公司的真实年报上，314 个人工核验数值中规则通道覆盖 293 个，错配为 0；流水线迭代后，两家公司深度研报的质检得分由 74、67 分提升到 90、88 分（同一评分标准、单次盲评，非对照实验）；评审模型重复评分的逐条一致率为 86% ~ 91%，判出问题经人工确认的精确率为 84%。所有数值计算均可在保存的输入上离线复算。

1　引言

卖方深度研报是二级市场投研最主要的文字产品：分析师读完年报、季报与公告，搜集行业与竞争资料，建立盈利预测和估值模型，最后写成二三十页、数十张图表的报告。一份年报点评加估值模型通常要花费分析师数天时间。大语言模型在长文档理解与写作上的能力，使“让模型直接读年报写研报”看起来触手可及。

然而在我们的早期实验中，模型直接生成的研报存在三类系统性失败，每一类都足以让整篇报告失去可信度：

* 共同第一作者。　† 通讯作者：陈沫樵（mochiaochen@gmail.com）。

2026 北京市大学生金融人工智能竞赛参赛作品（主题：金融投研智能体构建）。所有估值结论均依赖文中所列假设，不构成投资建议。

- **F1 数字幻觉**。模型会记错或编造财务数字，混淆合并报表与母公司报表、本期与上期、营业收入与营业总收入。即使数字抄对，也看不出它来自年报哪一页。
- **F2 计算不可靠**。同比增速、占比、单季拆分、三表勾稽和折现估值由模型“心算”时会出错，且结果随采样波动，无法复现，也无法做敏感性分析。
- **F3 外部信息无出处**。年报之外的行业规模、市场份额、一致预期、竞争对手动态，模型倾向于凭参数记忆补全，既可能过时，也可能是虚构。

第四个困难来自评价本身（**F4**）：研报的好坏涉及分析是否成立、论证是否充分、表达是否专业，长期依赖资深分析师的主观判断。没有可复核的评价，就无法判断系统改动是否真的带来改进，也无法驱动自动化的迭代。

本文的立场。 我们不试图让模型“更会算”，而是把计算从模型手里拿走。FinAgent 的总原则可以概括为一句话：**模型读、抄、写、判断；程序校验、计算、拦截**。模型给出的每一个数字，要么能在原文里逐字找到，要么由程序从真值表算出，要么登记在外部来源表中并在句中标注来源编号；否则会被程序拒收或拦下。在此基础上，研报的生产被拆成若干只靠文件交接、互不继承上下文的会话，每一步都有程序闸门；研报的评价被拆成程序判定的数据闭合与评审模型判定的分析质量两部分，评审模型本身也要先经过校准。

贡献。 本文的贡献如下：

1. **理解与计算分离的系统架构**（第 4、5 节）。我们把九条设计规范写成可由程序检查的不变量，并给出每条规范的执行机制与测试，使“每个数字可追溯”从口号变为流水线的硬约束。
2. **面向真实年报版式的证据约束抽取**（第 6 节）。针对 A 股年报中的稀疏网格、跨页续表、科目折行、同页多单位声明等版式问题设计规则通道，并与模型通道交叉校验；两通道冲突时记为问题交人工，不静默取舍。
3. **数字护栏与数字锁**（第 8 节）。护栏按每个数字的展示精度设定容差，只在结论性产物中找出处，从而压低“碰巧放行”的概率；数字锁保证改写只能删减、不能新增或改动数字；二者共同把 LLM 改写报告的风险限制在措辞与结构层面。
4. **先搜集、后分析的多会话研报流水线**（第 9 节）。五个角色（研究助理、分析师、主笔、复核编辑、排版设计师）在仓库之外的隔离工作区中顺序工作，11 道程序闸门逐阶段核对产物及其新鲜度。
5. **可校准的评测闭环**（第 10 节）。四层评分标准（硬闸门、数据闭合、分析质量、表达与合规）加负向条目；评分标准对生成方不可见，问题单按类型回传；我们用重复评审、程序制造缺陷和人工复核三组证据检验评审模型是否可信。

结果概览。 在贵州茅台、宁德时代、招商轮船三家 2025 年年报上，规则通道覆盖 314 个人工核验值中的 293 个（改进前为 128 个），错配为 0（第 11.3 节）。流水线在“先搜集外部资料”的基础上加入深度专题调研后，中际旭创与宇树科技两份研报的质检得分由 74、67 分提升至 90、88 分，均达到交付标准（第 11.5 节）。评审模型的校准显示：重复盲评逐条一致率 86%～91%、总分极差 7～8 分；无来源数字、机器腔、评级矛盾等缺陷能够判出，整节删除类缺陷的判出率较低，原因分析见第 11.6 节。我们同时如实报告这些数字的局限：样本量小、版本之间不是对照实验、评审与生成模型同族（第 12 节）。

2 相关工作

财报数值推理。 FinQA [3]、ConvFinQA [4] 与 TAT-QA [25] 构造了基于财报表格与文本的数值推理问答，FinanceBench [8] 进一步在真实上市公司公告上评测开放式问答。这些工作一致表明，即便是强模型，在多步数值推理和定位证据上也频繁出错。与问答不同，我们面对的是长文档生成：一份研报包含上千个数字，任何一个错都会损害整篇可信度。因此我们不追求提高模型的数值推理准确率，而是把数值推理整体移交给确定性程序，并在生成之后逐个数字核对。

程序辅助推理与工具调用。 PAL [5] 与 Program of Thoughts [2] 让模型生成程序、交给解释器执行；Toolformer [15] 与 ReAct [21] 让模型在推理过程中调用外部工具。这些方法把计算"交给工具"作为模型的可选行为。FinAgent 的区别在于把它变成不可绕过的约束：即使模型在文稿中自行写出一个算对的数，只要这个数不在本次运行的计算产物中，护栏就会拦下（第 8.1 节）。

金融大模型与金融智能体。 BloombergGPT [18] 与 FinGPT [19] 从预训练或微调角度提升模型的金融能力；FinMem [22] 与多模态交易智能体 FinAgent [23] 面向交易决策；FinRobot [20] 提出了面向金融分析的开源智能体平台。[1]这些工作主要关注模型能力或交易收益，较少讨论研报生产中的审计约束：数字来源、计算可复现、外部信息可追溯，以及如何客观评价一份长篇研报。

可归因生成与幻觉。 幻觉是自然语言生成的普遍问题 [9]。检索增强生成 [11] 与带引用的生成 [6] 通过检索证据并让模型标注引用来缓解幻觉，其可信度通常由自然语言推理模型或人工判断。我们的护栏面向数字这一更窄但更关键的对象，判定是确定性的：数字在指定精度下是否等于某个可追溯的值，指标与期间是否与真值表绑定一致。

以模型为评审与细则化评测。 MT-Bench 与 Chatbot Arena [24]、G-Eval [12]、Prometheus [10] 等研究了用模型充当评审的可行性与偏差；HealthBench [1] 等工作把专家知识写成逐条的评分细则，由模型逐条判定后按权重计分。已有研究也发现评审模型会偏好自己生成的文本 [14]。我们沿用"逐条二元判定 + 程序计分"的骨架，但把能由程序判的条目（数据闭合）全部交给程序，评审模型只判分析与表达；并通过程序制造已知缺陷来检验评审的灵敏度，这比只看评审间一致性更能发现"评审漏判"。

多智能体协作与自我修订。 AutoGen [17] 与 MetaGPT [7] 用多个角色化的模型协作完成复杂任务；Self-Refine [13] 与 Reflexion [16] 让模型根据反馈迭代改进。我们的编排器不是模型，而是确定性的 Python 状态机；各角色之间不共享对话，只通过工作区文件交接；评审标准对生成方不可见，回传的问题单只含理由与证据，以减轻迭代过程中对评分标准的过拟合。

3 任务定义与记号

输入。 核心输入是一份 A 股上市公司年报（PDF），可附季报、业绩预告、卖方一致预期、可比公司列表和一份待核查的研报草稿。对研报生成任务，还允许系统联网检索年报之后的公开信息，但检索结果必须登记（第 8.2 节）。

文档与片段。 解析器把文档 D 切成带锚点的片段序列 $\{c_k\}$，每个片段记录文本、表格行、所在页码、表格标题、单位提示，以及（若为跨页续表）它所接续的片段。片段是来源定位的最小单位。

[1]本系统的代码名 FinAgent 与 Zhang et al. [23] 的工作同名但无关，正式名称待定。

事实与真值表。 设指标集合为 $\mathcal{M}$（当前 49 个，涵盖利润表、资产负债表、现金流量表核心科目与主要比率），期间集合 $\mathcal{P}$ 包括年度 `2025`、半年度 `2025H1`、单季 `2025Q3` 与累计 `2025Q1–3`。一条**事实**是六元组

$$f = (m,\ p,\ v,\ r,\ u,\ \sigma), \qquad m \in \mathcal{M},\ p \in \mathcal{P}, \tag{1}$$

其中 v 是换算到基本单位后的数值（金额为元，比率为百分数值，如 14.49 表示 14.49%），r 是原文写法（如 `"1,234.50"`），u 是原文单位，σ = (文件, 页码, 片段, 行, 列) 是来源定位。程序派生的值（例如单季拆分）不带 σ，而带公式与所依赖的事实。所有事实与派生值构成**真值表** $T : \mathcal{M} \times \mathcal{P} \rightharpoonup f$。

任务 A：财报分析（赛题 2）。 给定 T，输出同比、环比、单季拆分、勾稽校验、异常信号、利润变动归因、实际与一致预期或业绩预告的对比，以及每股收益质量判断。所有数值都是 T 的确定性函数。

任务 B：估值建模（赛题 4）。 给定 T、经营数据 E（分产品量价、产能等，逐格核对过原文）与估值假设 Θ（预测参数、折现参数、参考股价及其来源），输出三情景三表模型、FCFF 折现估值、敏感性矩阵、可比公司估值、反向折现（股价隐含了什么）和程序评级。Θ 中每一项都标明为"假设"，并写出依据。

任务 C：研报纠错（赛题 5）。 给定研报草稿 R，抽取可核验的数字断言 $a = (m, p, \tau, x, r, u)$，其中 $\tau \in$ {水平值, 比率, 同比, 百分点变动}，并把每条断言判为

$$\{\text{正确},\ \text{数值错误},\ \text{单位错误},\ \text{期间错配},\ \text{口径错误},\ \text{方向错误},\ \text{无法核实}\},$$

同时给出真值、来源定位与修改建议。

研报生成。 在完成 A、B 之后，生成一份卖方体例的深度研报文稿 N 与排版后的 PDF。记 $\mathrm{Num}(N)$ 为 N 中可识别的"数字 + 单位"记号集合，V 为本次运行结论性产物中的数值集合，S 为外部来源登记表中的数值集合。我们要求

$$\forall\, x \in \mathrm{Num}(N):\quad \exists\, v \in V \cup S,\quad |x - \phi_u(v)| \le \delta(x), \tag{2}$$

其中 ϕ_u 把基本单位换算到 x 的展示单位，$\delta(x)$ 是由 x 的展示精度决定的容差（第 8.1 节）；若 v 只来自 S，则 x 所在的句子必须带对应的来源编号。式 (2) 是硬约束；研报的分析质量与表达质量是软目标，由第 10 节的评分标准度量。

单位约定。 财务事实内部统一用元；估值模块（三表、DCF、可比公司）用百万元，每股用元；比率存百分数值。同一页出现多个"单位："声明时，每张表取其上方最近的声明。这些约定在代码与技能说明中保持一致，护栏在比较时按约定换算。

4 设计规范

FinAgent 的开发由一组书面设计规范约束。规范的价值不在于表述本身，而在于每一条都对应一个能由程序检查的不变量、一个执行机制和至少一个测试；违反规范的改动会在测试或流水线闸门上失败。表 1 汇总了九条核心规范，本节其余部分解释其中几条背后的取舍，以及流水线层面另外五条决策。

4.1 为什么连约数也不许心算

P1 最直观的违反是模型写错增速，但更常见、也更难发现的是"约三倍""超过四成"这类约数。它们看起来无害，读者却会把它们当作事实引用。我们的规则是：如果约数

表 1 核心设计规范、对应的不变量与执行机制。

规范	不变量	执行机制
P1 LLM 不做算术	文稿中的任何派生数（增速、占比、约数）都必须出现在本次运行的计算产物中	数字护栏 `guard`
P2 每个数字可追溯	抽取事实带 σ；模型抄的数字必须按数字边界出现在所写页码的原文中；外部数字的摘句必须包含该数字	`find_evidence`、`verify-facts`、`verify-sources`
P3 双通道不静默取舍	规则通道与模型通道取值不一致时生成问题记录，不得只保留一方而不留痕	`merge()`；抽取闸门
P4 一张真值表	估值与纠错只读 T；抽取存在拒收、冲突、勾稽失败或核心缺失时，流水线在估值前停止	流水线阻断
P5 全程追踪	每次文件读取与产物写出记录 SHA-256；每步、每次工具调用与模型调用写入事件流	`RunTracer`
P6 单位约定	事实用元、估值用百万元、比率用百分数值	引擎与护栏统一换算
P7 误报控制优先	纠错容差取展示精度的半个末位；错误类型必须找到精确的替代解释才下结论；预测与缺真值断言判“无法核实”	`verify_claim`
P8 事实 → 推论 → 观点	报告按“披露事实 / 程序推论 / 作者观点”分层，估值写明假设、适用范围与风险	写作技能；出处分类
P9 LLM 输出加护栏	任何让模型改写报告的功能都必须加数字锁	`numlock`

有价值，就让程序算出精确值并写入产物，文稿引用产物；如果没有价值，就删掉。这条规则让护栏可以对文稿中的每个数字一视同仁，不需要区分“精确数”与“示意数”。

4.2 为什么双通道冲突要交人工

抽取由模型按页读原文完成，规则通道独立取数做交叉校验。这样的设计面临一个诱惑：冲突时自动采信“更可信”的一方。我们拒绝这样做，因为冲突本身就是最有价值的信号——它往往意味着合并报表与母公司报表混用、续表表头错配、单位声明识别错误。P3 要求冲突以问题记录的形式进入抽取闸门，阻断后续估值，直到有人（或模型回到原文）解释清楚。

4.3 为什么误报控制优先于召回

研报纠错的误报代价远高于漏报：一个误报会让使用者怀疑整份核查结果，并且需要人工逐条排除。P7 因此规定：容差按研报展示精度计算，例如“58.62 亿元”的容差是 ± 0.005 亿元，四舍五入不算错；错误类型（单位、期间、口径、方向）只有在找到一个精确匹配的替代解释时才能下结论，否则一律归为“数值错误”；预测性表述与缺少真值的断言判为“无法核实”，不计为错误。

4.4 流水线层面的五条决策

上述规范面向计算引擎。研报生成流水线在此基础上另有五条决策，它们来自对早期产物的人工质检：

D1 先搜集、后分析

早期研报中，行业与竞争章节大量出现模型凭记忆写出的数字。我们把外部资料搜集提前为第一个阶段，由独立的研究助理会话完成并登记来源；后续会话只能引用已登记的外部事实。

D2 上下文隔离、只靠文件交接

写作、复核、评审若共享对话，复核者会继承写作者的思路，评审者会看到写作过程而变得宽容。每个角色是一次独立的无头会话，只读写工作区文件；工作区建在仓库目录树之外，看不到其他公司的产物。

D3 评分标准对生成方不可见

评分标准一旦进入生成方的上下文，系统就会朝评分条目优化，而不是朝更好的研报优化。评分标准与完整评测结果保存在工作区之外，回传给生成方的问题单只含理由与证据，不含条目原文与分值。

D4 不预设行业框架

技能说明只给方法与约束（例如“核心驱动至少两环定量推导”），不给“白酒看吨价、电池看 GWh”这类行业模板。模型读完年报与资料包后自行推导该公司的收入与利润驱动。预设模板会让不同公司的报告长得一样，也会让模型在模板不适用时硬套。

D5 文风不做程序拦截

我们曾考虑用正则规则拦截“不是……而是……”“相互印证”等机器腔句式，最终放弃：同样的意思换个说法照样是机器腔，规则只会催生规避。文风交给一个没有参与写作的复核编辑会话改写，数字锁保证它只能动措辞和结构。

5 系统总览

图 1 展示了系统的四层结构。分工原则贯穿各层：模型负责读、抄、写、判断；程序负责校验、计算、拦截。

计算引擎。 引擎只依赖 Python 标准库与 PDF 解析库 pdfplumber，提供三条固定流水线：财报分析（解析 → 抽取 → 分析 → 经营数据 → 业绩对比 → 报告）、估值（三表 → 估值计划 → DCF → 情景 → 预测摘要 → 可比公司 → 估值报告）与研报纠错（解析研报 → 抽断言 → 核查）。每一步通过 `Pipeline.step` 注册，产物写入 `runs/<run_id>/` 并记录哈希。引擎另提供一组供模型调用的校验命令：`locate`（定位关键页、母公司报表页、单位与扫描页）、`verify-facts`、`verify-extra`、`verify-sources`、`guard` 与 `numlock`。

技能层。 技能（skill）是写给模型的工作说明书，包括步骤、约束、命令与反例。四份技能各管一类角色：`finagent-research`（抽取、校验、估值、纠错）、`finagent-report`（资料搜集与深度调研方法、分析框架推导、文稿模板与排版工具包 reportkit）、`finagent-review`（复核编辑的改写边界与对账方法）、`finagent-pipeline`（编排入口，本身不写研报）。

研报流水线。 `report_pipeline` 为单家公司在仓库之外搭建隔离工作区，依次启动五个无头会话，并用 11 道闸门核对每个阶段的产物（第 9 节）。



图 **1**　FinAgent 的四层结构。自下而上：**计算引擎**（确定性 Python，抽取校验、真值表、三表、DCF、可比公司、护栏与数字锁，全程追踪哈希）；**技能层**（规定模型在各角色中“怎么做”的四份技能说明）；**研报流水线**（隔离工作区、五个互不继承上下文的会话、11 道程序闸门）；**质检循环**（程序检查与盲评、问题单路由、回滚到最佳轮次）。蓝色为确定性组件，橙色为模型会话，红色为闸门。

质检循环。 `refine` 在全部闸门通过后启动独立的盲评会话，按评分标准逐条判定并由程序计分；未达交付标准时把问题单按类型回传给相应角色，最多迭代三轮并回滚到得分最高的一轮（第 10 节）。

5.1　模型与引擎的接口

模型与引擎之间只有一种交互方式：模型会话在工作区里读文件、写文件，并以命令行调用引擎；引擎读模型写下的文件，给出校验结果或计算产物。模型从不直接写真值表、估值结果或评分，引擎也从不生成文字。表 2 列出了这种交接在各环节的具体形式。

表 **2**　模型会话与计算引擎之间的交接。

环节	模型写下	引擎给出
抽取	`facts.json`：指标、期间、原文写法、页码	`verify-facts`：逐字回查、规则通道比对、勾稽；通过后生成真值表
经营数据	`extras.json`：逐格抄录的表格，派生比率只写声明	`verify-extra`：逐格回查，比率由程序计算
外部资料	`sources.json`：检索词、网址、日期、摘句	`verify-sources`：结构与“数字在摘句中”检查
估值	`valuation_assumptions.json`：参数与依据	三表、DCF、可比公司、反向 DCF、程序评级
纠错	`claims.json`：从研报拆出的数字断言	逐条核查与错误类型诊断
文稿	`narrative.md`	`guard`：出处、绑定、来源编号；`numlock`：改写前后数字



图 2　证据约束的双通道抽取。规则通道从白名单表格中按科目匹配取数；模型通道（分析师会话按页读 PDF）给出原文写法与页码。程序逐字回查原文、合并两通道、做勾稽，任何拒收、冲突、勾稽失败或核心缺失都会让流程回到抽取（最多三轮），通过后才生成真值表。

5.2　追踪与复现

所有输入文件都经由 `tracing.read_file()` 读取，所有产物经由 `tracer.write_artifact()` 写出。每次运行生成 `events.jsonl`（逐条记录文件读取、工具调用、模型调用、产物与备注，大对象截断并保留哈希）与 `manifest.json`（输入与产物哈希、配置、命令行参数、版本与耗时）。研报流水线的每个会话另把模型事件流保存在 `.pipeline/agents/*.jsonl`。因此复现分为两层：已保存输入上的全部程序计算可以离线逐位复算；重新生成文稿或重新盲评则需要相应的模型环境，我们不宣称能离线复现模型输出。

6　证据约束的抽取与真值表

抽取是整个系统的地基：真值表中的一个错数会沿着分析、估值、文稿一路传播。图 2 给出了抽取流程。本节按顺序介绍页定位、面向真实年报版式的解析、两条抽取通道、合并规则与校验闸门。

6.1　页定位

一份 A 股年报通常有 200 页以上，三大报表和主要会计数据只占其中十几页。`locate` 扫描全文，给出建议阅读页、各页的单位声明、母公司报表所在页（需要回避），并标出没有文字层的扫描页。扫描页目前直接拒收，不做光学字符识别：与其引入一个错误率未知的识别环节，不如明确告知输入不合格。

6.2　真实年报版式的解析

在自造样例上，表格解析几乎没有难度；换到真实年报，规则通道起初只能取到 314 个核验值中的 128 个。逐一排查后，我们归纳出四类版式问题并分别处理：

- **稀疏网格。**部分年报（如宁德时代）的表格在 PDF 中被切成远多于实际列数的网格，科目与数字落在不同的空列里。`compact_columns` 合并稀疏列：从左到右把相邻列归为一组，只要组内任一行两列都有值就另起一组；规整的表格没有全空列，原样返回。
- **跨页续表。**资产负债表等长表跨页时，续页没有表头，规则通道无法判断各列对应的期间。`link_continuation` 判断页首无标题的表格是否为上一页表格的续表：若是，

则沿用首页表头的期间列，并记录 `continued_from`。我们只记录续表关系、不合并两页的行，因此每一行的页码仍是它实际所在的页，来源定位不失真。

- **科目折行。**“归属于上市公司股东的净利润”这类长科目会折成两行，数字只在第一行。`merge_wrapped_labels` 把只有标签、没有数值、且满足折行特征（上一行括号未闭合、上一行写满列宽、或本行以不会作科目开头的字开头）的行接回上一行；以冒号结尾的小节标题不算折行。
- **同页多单位。**同一页可能先后出现“单位：元”与“单位：万元”。解析器记录各“单位：”声明的纵向位置，每张表取其上方最近的声明；表格标题只在上一张表格底部以下、最多向上六行范围内查找。

6.3 规则通道

规则通道只从白名单表格取数：主要会计数据、分季度主要财务指标、非经常性损益、三大报表及其补充资料。真实年报附注里有大量与报表科目同名但口径不同的明细表（受限资产、费用明细、关联方往来），混入会造成冲突，因此被排除。具体规则包括：标题含“母公司”的表格不取（只取合并报表）；首行含“关联方”的表格不取；表头含“比例”“比重”“占”的结构比列不当作期间列；同表既有“营业总收入”又有“营业收入”时（含金融子公司利息收入等情形），营业收入取后者；现金流量表补充资料中的调节项（如“财务费用（收益以‘-’号填列）”）不当作报表科目本身。财报自带的“本年比上年增减”列也被保留，用于与程序计算的同比交叉核验；比率类指标（如净资产收益率）的增减按百分点核对，而不是相对增速。

6.4 模型通道与证据校验

分析师会话按 `locate` 的建议逐页阅读，写出 `facts.json`：每条事实给出指标、期间、原文写法与页码。

模型给出的每个数字都要经过证据校验（`find_evidence`）：设 P 为该条所写的页码，$\nu(r)$ 为原文写法 r 去掉千分位、括号、百分号与正负号后的数字串，则要求该页至少有一个片段（表格优先）的文本按**数字边界**包含 $\nu(r)$，

$$\exists\, c_k :\ \mathrm{page}(c_k) = P \ \wedge\ \nu(r) \text{ 以完整数字的形式出现在 } \mathrm{text}(c_k) \text{ 中}, \tag{3}$$

即前后不能紧邻其他数字或小数点，避免“1.23”命中“11.234”这类部分匹配。此外，期间格式不合法、口径不是合并报表、单位不合法、缺页码的条目同样拒收。否则该条被拒收并记录原因。这条规则简单到近乎粗暴，却有效地排除了模型“算出来的”或“记忆中的”数字：模型可以抄错位置，但很难凭空写出一个恰好出现在所引页上的数字。

6.5 合并：不静默取舍

算法 1 给出合并规则。同一通道内多处取到同一 (m, p) 时，取值相同则互为佐证，不同则生成高严重度问题；两通道之间取值相同标记为 `rule+llm`，不同则生成中严重度问题，并注明暂时保留的是模型读原文的结果。无论保留哪一方，问题记录都会进入抽取闸门。

6.6 校验闸门与勾稽

`verify-facts` 把合并后的事实送入分析模块，再汇总四类阻断问题：拒收（证据校验失败）、冲突（算法 1 的问题记录）、勾稽失败与核心指标缺失。勾稽包括：

算法 1 双通道合并（merge）

Require: 规则通道事实 F_R，模型通道事实 F_L，冲突时的保留方 $\pi \in \{\text{rule}, \text{llm}\}$

$M \leftarrow \emptyset$，$I \leftarrow \emptyset$
for $f \in F_R$ **do**
  if $(m,p) \notin M$ **then** $M[(m,p)] \leftarrow f$
  else if close$(M[(m,p)].v, f.v)$ **then** 记 $f.\sigma$ 为佐证
  else $I \leftarrow I \cup \{$高：同一指标多处取值不一致$\}$
  end if
end for
for $f \in F_L$ **do**
  if $(m,p) \notin M$ **then** $M[(m,p)] \leftarrow f$
  else if close$(M[(m,p)].v, f.v)$ **then** $M[(m,p)]$.method $\leftarrow$ rule+llm
  else $I \leftarrow I \cup \{$中：规则与模型不一致（注明保留方）$\}$；**if** π = llm **then** $M[(m,p)] \leftarrow f$
  end if
end for
return M, I ▷ I 非空即阻断估值

- 资产总计 = 负债和所有者权益总计；负债和所有者权益总计 = 负债合计 + 所有者权益合计；所有者权益合计 = 归母权益 + 少数股东权益（相对容差 10^{-6}，且不小于 1 元）；
- 非经常性损益合计 = 归母净利润 − 扣非归母净利润（仅当非经常性损益为财报列示值时核对）；
- 财报列示的同比增减与程序计算的同比在展示精度内一致；
- 单季拆分：$Q_2 = H_1 - Q_1$，$Q_3 = Q_{1\text{-}3} - H_1$，$Q_4$ = 全年 − $Q_{1\text{-}3}$；当某个单季数既有财报列示值、又能由累计数相减得出时，二者必须一致。程序记录每个单元格依赖的列示单元格集合，避免“自己和自己比”。

有任何阻断问题时命令返回非零，模型须回到原文核对修正，最多三轮。只有全部通过，才会生成 `facts.verified.json` 并进入估值。

6.7 经营数据

估值需要的分产品量价、区域收入、产能、分红等数据不在三大报表里。模型按分析框架中的驱动清单把相关表格逐格抄进 `extras.json`，`verify-extra` 逐格回查原文；任何派生比率（吨价、增速、差值）不允许模型自己写，只能声明为 `ratios`，由程序计算，并支持别名以便正文引用。

7 确定性的分析与估值

真值表生成之后，所有数值计算都由确定性 Python 完成。模型在这一层的作用只有两个：起草估值假设（每一项写出依据并标为“假设”），以及之后在文稿中解读产物。

7.1 财报分析

同比与环比变动统一定义为

$$g = \frac{x_t - x_{t-1}}{|x_{t-1}|} \times 100, \tag{4}$$

分母取绝对值，以符合 A 股对基期为负时的惯例。异常信号由固定规则产生，并附严重度与证据定位：非经常性损益占归母净利润比例过高（S1）；归母净利润增速显著高



图 3 估值链。经营数据驱动分部预测（销量 × 单价 → 收入，毛利率与费用率），分部预测驱动三表联动模型，DCF 的全部情景向量取自三表模型的同一情景；企业价值经逐项列示的股权价值桥得到每股价值，评级由程序按上行空间给出。虚线表示逐期一致性核对。

于扣非增速（S2）；净现比偏低（S3）或净利润增长而经营现金流下降（S4）；应收账款、存货增速显著快于收入（S5、S6）；毛利率大幅变动（S7）等。利润变动归因桥把归母净利润的变动拆到毛利、各项费用、投资收益、减值、所得税与少数股东损益上。业绩对比模块给出实际值与一致预期的超预期或不及预期判断、与业绩预告区间中值的比较，以及每股收益质量筛查。

7.2 估值链：分部预测驱动三表，三表驱动 DCF

图 3 给出估值链。设公司有分部 $s = 1, \dots, S$，预测期 $t = 1, \dots, N$。

分部预测。 分部收入由量价驱动，

$$\text{Rev}_{s,t} = Q_{s,t} \cdot P_{s,t}, \qquad Q_{s,t} = Q_{s,t-1}(1 + g^{Q}_{s,t}), \quad P_{s,t} = P_{s,t-1}(1 + g^{P}_{s,t}), \tag{5}$$

分部成本由分部毛利率决定，再汇总为公司收入增速与综合毛利率。基期分部收入加其他收入必须等于报表营业收入（评分条目 L1-02）。没有量价数据的行业（例如航运），可以直接给出合并收入增速与毛利率路径，并在分析框架中说明原因。

三表联动。 三表模型按收入、毛利率、费用率、营运资本周转、资本开支与折旧、分红率等驱动预测利润表、资产负债表与现金流量表。经营费用作为轧差项（毛利 − EBIT − 折旧摊销），保证历史期 EBIT 与 DCF 口径一致。模型逐期检查资产负债表平衡、现金流与现金余额衔接以及各项滚动（固定资产、债务、权益）。这里的 EBIT 定义为

$$\text{EBIT} = \text{利润总额} + \text{财务费用} - \text{投资收益} - \text{公允价值变动收益}, \tag{6}$$

即剔除投资类收益；被剔除收益对应的投资类资产在股权价值桥中作为非经营性资产加回。

FCFF 折现。 DCF 的收入、EBIT、折旧摊销、资本开支与营运资本**逐期取自三表模型的同一情景**（评分条目 L0-03 逐期核对其偏差），假设文件中的利润率等字段不再单独

生效。

$$\mathrm{FCFF}_t = \mathrm{EBIT}_t(1-\tau_t) + \mathrm{DA}_t - \mathrm{Capex}_t - \Delta\mathrm{NWC}_t, \tag{7}$$

$$\mathrm{WACC} = w_E K_E + w_D K_D(1-\tau) + w_P K_P, \qquad K_E = r_f + \beta \cdot \mathrm{ERP} + \text{规模、特定与国别溢价}, \tag{8}$$

$$\mathrm{TV} = \frac{\mathrm{FCFF}_N(1+g_\infty)}{\mathrm{WACC} - g_\infty} \quad \text{或} \quad \mathrm{TV} = \mathrm{EBITDA}_N \times \text{退出倍数}, \tag{9}$$

$$\mathrm{EV} = \sum_{t=1}^{N} \frac{\mathrm{FCFF}_t}{(1+\mathrm{WACC})^{t-0.5}} + \frac{\mathrm{TV}}{(1+\mathrm{WACC})^{N}}. \tag{10}$$

现金流按年中折现，终值按第 N 年末折现。股权价值桥逐项列示：

$$\begin{aligned} \text{股权价值} = \mathrm{EV} &+ \text{现金} + \text{非经营性资产} + \text{联营投资} \\ &- \text{有息负债} - \text{租赁负债} - \text{少数股东权益} - \cdots, \end{aligned} \tag{11}$$

每股价值 = 股权价值 / 稀释后总股本。非经营性资产须逐项核对年报原文。三个情景（悲观、基准、乐观）各自对应三表模型的一组驱动；敏感性矩阵覆盖 WACC × 永续增长率等组合，程序检查其单调方向（WACC 上升价值下降、永续增长上升价值上升）。

可比公司与交叉验证。 可比公司估值逐家计算企业价值桥与倍数，处理负值与无意义倍数，按四分位距剔除离群值，给出 P25 ~ P75 的估值区间；另输出利润率对比页，用于判断倍数差异是否有基本面支撑。

反向 DCF：股价隐含了什么。 正向 DCF 由假设推出价值；反向 DCF 固定其余假设为基准情景，只动一个变量 θ（预测期收入增速整体平移、EBIT 利润率整体平移、WACC 平移、永续增长率或退出倍数），用网格扫描加二分法求解

$$V_{\text{每股}}(\theta) = P_{\text{参考}}, \tag{12}$$

得到参考股价隐含的取值，并与基准假设和五年历史区间对照分档。各变量单独求解而非联合求解，且只作用于 DCF 层、不回推三表模型；这些口径简化写入产物说明。反向 DCF 为研报的“预期差”一节提供了可以复算的锚点。

程序评级。 评级不由模型给出，而由程序按基准每股价值相对参考股价的上行空间 $u = V_{\text{base}}/P_{\text{参考}} - 1$ 决定：

$$\text{评级} = \begin{cases} \text{买入} & u \geq 20\% \\ \text{增持} & 5\% \leq u < 20\% \\ \text{中性} & -5\% \leq u < 5\% \\ \text{减持} & -20\% \leq u < -5\% \\ \text{卖出} & u < -20\% \end{cases} \tag{13}$$

研报只引用这一结果。这样评级与估值之间不会出现模型“看好公司所以给买入”的漂移（评分条目 L0-05）。

7.3 估值模型工作簿

估值结果同时输出为按投行体例排版的工作簿：输入单元格蓝色、公式黑色、跨表引用绿色；分部预测 → 三表假设（含情景开关）→ 三表全公式联动与勾稽检查 → DCF，



图 4 数字护栏与数字锁。左：护栏把文稿中的每个“数字 + 单位”按单位与展示精度换算，到本次运行的结论性产物与外部来源登记中找出处；找不到则列为“无出处数字”，指标或期间配错则列为“绑定不符”，只在来源登记中有出处却未标编号则列为“缺引用”。右：数字锁比较改写前后的数字集合，只允许减少，不允许新增或改动。

历史财务、分季度与经营数据各占一页。关键计算以公式而非数值写入，使用者可以改假设看结果。工作簿审计模块检查跨期公式孤岛（判失败）、公式内写死的数字与易变函数（提示）。

7.4 与参照引擎的一致性

三表、情景表、业绩对比、DCF 与可比公司引擎的计算口径参照了一个外部投研插件（细节与许可证说明见仓库 `THIRD_PARTY.md`），但实现全部自研、仓库中不包含其代码。一致性由两类测试保证：`tests/test_golden.py` 要求自研引擎在样例上的产物与改造前原引擎的快照完全相等；在本地存在原插件时，`tests/test_parity_reference.py` 用样例、边界值与数百组随机输入逐项对照（三表浮点逐位相等，DCF 相对误差小于 10^{-9}）。有意修改口径时同步更新快照，并在变更说明中写明原因。

8 数字护栏、数字锁与研报纠错

抽取与计算可以做到确定，但文稿终究要由模型来写。本节介绍把式 (2) 落到实处的几道程序检查：数字护栏（文稿里的数字有没有出处）、指标绑定（数字是否配对到了正确的指标与期间）、外部来源登记、出处分类、数字锁（改写有没有动数字）和内部一致性检查。研报纠错（赛题 5）与护栏共用同一个核查引擎，放在本节最后介绍。

8.1 数字护栏

记号化。 护栏用一个正则表达式从文稿中抽取“数字 + 单位”记号 $x = (r, u)$，单位包括亿元、万元、元、元/股、元/吨、%、个百分点、倍等。年份（19xx、20xx）、一位整数序号、“第 n 页 / 节”这类不带单位的记号不检查。

出处集合。 出处值集合 V **只取结论性产物**：真值表与分析结果、业绩对比、经营数据（按列单位换算）、DCF 结果、可比公司结果、预测摘要、分部预测、估值计划中的 WACC 与终值参数、股权价值桥各项、以及纠错结论中的正确值。估值产物以百万元计，护栏另按 $\times 10^6$ 换算。两类数值被有意排除在 V 之外：

- **研报草稿中的断言原文。**草稿里的错误数字不能成为出处，否则纠错引擎看过的错数会反过来为文稿背书。

- **中间网格**。三表逐期明细与敏感性矩阵包含成千上万个中间值，把它们纳入 V 会显著提高编造数字“碰巧”命中的概率。

按展示精度的容差。 设记号 x 的原文写法有 d 位小数，单位换算因子为 ϕ_u（亿元为 10^{-8}，万元为 10^{-4}；百分数额外允许以小数存储的比例，即因子 $\{1, 100\}$），则

$$\mathrm{pass}(x) \iff \exists\, v \in V,\ \exists\, \phi \in \Phi_u :\ \big||v| \cdot \phi - |x|\big| \le \tfrac{1}{2}\, 10^{-d}\,(1 + 10^{-9}). \tag{14}$$

也就是说，“58.62 亿元”只与真值除以 10^8 后四舍五入到两位小数相比较，“14.49%”只与百分数值比较。容差随展示精度收紧，是护栏能压低误放行的关键：若把一个编造的数字视为在宽度为 W 的区间内随机取值，而 V 中落在该区间的值有 n 个，则它碰巧通过的概率粗略为 $n \cdot 10^{-d}/W$。精度越高、V 越小，误放行越少；这也是我们宁可把中间网格排除在 V 之外的原因。

指标绑定。 式 (14) 只回答“这个数有没有出处”，回答不了“这个数是不是配给了对的指标与期间”——模型完全可能把营业成本的数写成营业收入。护栏因此把文稿当作一份研报，送入纠错引擎（第 8.6 节）按“指标 + 期间”逐条核对。为避免误报，只对口径明确的断言做绑定核对：预测表述、季度与半年度数、跨年序列、分部数据、现金流明细、“由 a 升至 b”这类区间表述一律跳过（它们仍要通过式 (14) 的出处核对）；分部表述（例如“某产品毛利率”）若句中点名了经营数据表的某一行、且数值等于该行某个已核对的值，则按分部数据核对通过。

8.2 外部来源登记

年报之外的事实只能来自外部来源登记表 `research/sources.json`。每条来源记录标题、发布方、网址、发布日期与访问日期，以及若干条原文摘句（不超过 120 字）；摘句中声明的每个数值都必须作为子串出现在摘句里——这与模型抽取“数字须出现在所写页码的原文中”是同一原则。每个检索问题还记录所用检索词与是否找到。护栏把登记表中的数值并入出处集合，但附加一条规则：**只能在外部来源中找到出处的数字，所在句子必须带对应的 `[S 编号]`**，否则列为“缺引用”。

8.3 出处分类：事实、推论与观点

赛题要求区分事实、推论与观点。我们把这一要求落到每个数字上：护栏按产物给出处值打标签——真值表中按抽取方式分为“报告披露”与“程序计算”，可比公司原始数据与来源登记为“外部来源”，DCF 与预测为“模型预测”，WACC 与终值参数为“估值假设”。文稿中每个有出处的数字归入

$$\text{事实} = \{\text{报告披露}, \text{外部来源}\},\quad \text{推论} = \{\text{程序计算}\},\quad \text{观点} = \{\text{模型预测}, \text{估值假设}\},$$

同一数字在多处有出处时优先认作事实。研报附录据此写出全文数字的事实、推论、观点构成。打标签不改变护栏的判定结果；我们在 24 次真实运行上核对过，带标签的出处集合与原出处集合逐值一致。

8.4 数字锁

复核编辑会话会删句、调序、改措辞。设 $\mathrm{Tok}(\cdot)$ 为去掉注释与来源标记后的“数字 + 单位”记号多重集，$\mathrm{Src}(\cdot)$ 为来源编号集合。数字锁要求

$$\mathrm{set}(\mathrm{Tok}(N_{\text{改写}})) \subseteq \mathrm{set}(\mathrm{Tok}(N_{\text{初稿}})) \quad\wedge\quad \mathrm{Src}(N_{\text{改写}}) \subseteq \mathrm{Src}(N_{\text{初稿}}). \tag{15}$$

算法 2 断言核查与错误类型诊断（`verify_claim`）

Require: 断言 $a = (m, p, \tau, x, r, u)$，真值表 T

if a 为预测性表述 **then return** 无法核实
end if
$t \leftarrow T.\text{value}(\tau, m, p)$；**if** t 不存在 **then return** 无法核实
$\epsilon \leftarrow \frac{1}{2} 10^{-d(r)} \phi_u$ ▷ 按展示精度的半个末位
if $|x\phi_u - t| \le \epsilon$ **then return** 正确
end if
if τ = 水平值且 $\exists u' \in \text{Alt}(u) : |x\phi_{u'} - t| \le \epsilon'$ **then return** 单位错误
end if
if $\tau \in \{$ 同比, 百分点 $\}$ 且 $|-x - t| \le \epsilon$ **then return** 方向错误
end if
if $\exists p' \ne p : |x - T.\text{value}(\tau, m, p')| \le \epsilon$ **then return** 期间错配
end if
if $\exists m' \in \text{Group}(m) \setminus \{m\} : |x - T.\text{value}(\tau, m', p)| \le \epsilon$ **then return** 口径错误
end if
return 数值错误（附相对偏差与修改建议）

数字可以减少，不能新增或改动；来源编号同理。数字锁只看记号本身，数字与指标的配对仍由改写后重跑的护栏核对。需要补数字或改结论的问题，复核编辑不能自己动手，只能写进人工复核清单，或在质检循环中路由回分析与写作侧（第 10.4 节）。数字锁让我们可以放心地给编辑会话很大的改写自由度：它能改变的只是措辞与结构。

8.5 内部一致性

程序只能核对单个数字的出处，核对不了“摘要写买入、正文写卖出”这类前后矛盾。一致性检查从文稿中抽取评级、各情景每股价值、预测年度公司整体的收入、利润、每股收益、毛利率及其增速，**只有实体、口径、期间、单位、情景都相同时才比较**，两处数值超出展示精度即判为矛盾；外部来源、条件句、敏感性与区间表述、分部口径不进入比较。在七份真实文稿上它零误报，对“摘要评级反转”的人为缺陷全部命中，因此评分标准 v2 把“数字前后矛盾”从评审模型判定改为程序判定。

8.6 研报纠错（赛题 5）

纠错时，模型会话先从研报中拆出可核验的数字断言（规则抽取作为补充与交叉检查），每条断言绑定指标、期间、类型、数值与单位。随后按算法 2 判定。

两个细节体现了“误报控制优先”。其一，错误类型诊断只在**找到精确匹配的替代解释**时才下结论：断言值换成另一单位、相反方向、另一期间或同组另一指标之后恰好等于真值。找不到时归为“数值错误”，而不是猜一个类型。其二，口径诊断只在容易混用的相关指标组之间进行，例如 {净利润, 归母净利润, 扣非归母净利润, 利润总额, 营业利润}、{营业收入, 营业成本}、{毛利率, 净利率}、{总资产, 总负债, 所有者权益, 归母权益}，避免跨指标的巧合匹配造成误判。每条结论都附真值、来源定位（派生值附公式与依赖的事实）和修改建议。

9 多会话研报流水线

前几节的引擎回答了“数字怎么算对、怎么不被写错”。本节回答另一个问题：怎样让模型写出一份有深度、像分析师写的研报。我们的答案是把研报生产拆成五个角色、十一个阶段，用确定性的编排器和程序闸门把它们串起来（图 5）。



图 5 研报流水线。确定性编排器在仓库目录树之外为单家公司搭建隔离工作区，依次启动五个互不继承上下文的会话：研究助理（阶段 1）→ 分析师（阶段 2 ~ 7）→ 主笔（阶段 8）→ 复核编辑（阶段 9）→ 排版设计师（阶段 10 ~ 11）。会话之间只通过工作区文件交接；每个阶段结束都运行 `status`，由 11 道程序闸门判定能否进入下一阶段。

9.1 隔离工作区

`report_pipeline build` 为一家公司在仓库目录树之外（默认 `~/finagent-samples/<公司>_<年度>`）建立工作区，只放入四样东西：三份技能说明、计算引擎（不含评测模块 `finagent/evaluation/`）、这家公司的年报，以及人提供的外部输入（例如参考股价及其来源）。

隔离的理由来自实际教训：仓库里有其他公司的抽取文件、分析框架、文稿和交付物，模型在仓库里工作时会照抄别家公司的框架，并浪费大量上下文去读无关文件；同时仓库里有评分标准，放在生成方能读到的地方就违背了决策 D3。工作区的权限配置因此做了两道围栏：**禁止读取**仓库所在目录及其上级目录；**禁止修改**引擎、技能说明、年报原件与工作区说明。允许的工具调用以白名单形式在启动无头会话时传入，包括以 `./py -m finagent` 开头的引擎命令、对 `data/` 的读写、联网检索与抓取等。

9.2 五个角色

表 3 列出五个会话及其默认模型。每个会话由一段简短的角色提示启动（附录 C 给出要点），具体做法写在技能说明和工作区说明（`CLAUDE.md` / `AGENTS.md`）里。会话之间不继承上下文；每个会话完成自己的阶段后运行 `status`，闸门通过即停止并汇报，由编排器决定是否启动下一个会话。已经通过的阶段在重跑时自动跳过。

9.3 十一道闸门

闸门由 `status` 统一判定，全部是程序检查（表 4）。三条横向规则使闸门难以被“绕过”：

1. **新鲜度**。校验结果必须晚于被校验文件的最后修改时间：例如 `facts.json` 改过而未重新校验，阶段 2 判未通过；估值运行早于任何输入文件的修改，阶段 7 判为“已过期”；PDF 早于文稿或排版脚本，阶段 10 判未通过。
2. **核对的是当前稿**。阶段 8 要求护栏结果对应的文稿与当前文稿逐字相同，并且护栏带上了外部来源登记。

表 3 五个生产会话。生成用 Opus 5.5，复核用 Sonnet 5.5；均可通过环境变量替换，执行器也可换成 Codex。

会话	模型	阶段	职责与边界
研究助理	Opus	1	联网搜集年报之后的信息，登记 `sources.json`、写资料包 `dossier.md`；不做财务抽取、不写分析框架
分析师	Opus	2 ~ 7	财务抽取与校验、分析框架（含深度专题）、深度调研、经营数据、估值假设、估值运行；不写文稿
主笔	Opus	8	读框架、资料包、专题笔记与运行产物，写“年报点评 + 深度研究”文稿，过护栏与篇幅闸门
复核编辑	Sonnet	9	没有参与写作；只改措辞与结构，过数字锁；拿不准但重要的事项写入人工复核清单
排版设计师	Opus	10 ~ 11	文稿已定稿，不改正文；写本公司的排版脚本、渲染 PDF、逐页检查，写交付说明

表 4 十一道程序闸门及其阈值（常量定义于 `report_pipeline.py`）。

阶段	通过条件
1 资料搜集	七个类别全部覆盖；资料搜集检索 ≥ 20 次；近 90 天内发布的来源 ≥ 3 个；`verify-sources` 无问题
2 财务抽取	拒收、冲突、勾稽失败均为 0，且校验晚于修改
3 分析框架	`framework.md` 含“外部调研清单”,“深度专题”≥ 3 个
4 深度调研	专题笔记 ≥ 3 份，每份 ≥ 1500 字且 ≥ 3 个独立来源；补充检索问题 ≥ 8 个；来源表晚于分析框架
5 经营数据	`verify-extra` 拒收 0 格
6 估值假设	有参考股价及其来源；依据中引用了资料包的 `[S 编号]`
7 估值运行	运行晚于全部输入，且带上了经营数据与资料搜集得到的可比公司表
8 文稿	护栏（含来源）通过且核对的是当前稿；正文 ≥ 15,000 字，深度研究占比 ≥ 60%
9 复核改写	初稿、修改记录与复核清单齐全；数字锁通过
10 排版	PDF 晚于文稿与脚本；25 ~ 32 页；图表 ≥ 60 个；已按公司与日期命名
11 交付说明	`delivery.md` 存在

3. **前序阻断**。只要前序阶段未通过，后续阶段即使文件齐全也一律判为未通过，避免拿过期产物当成果。

需要强调的是，篇幅、页数、图表数与来源数都是**数量下限**，不是质量认证；研报质量由第 10 节的盲评兜底。数量闸门的作用是阻止模型用“写短一点”来逃避深度调研。

9.4 先搜集、后分析

资料搜集阶段要求覆盖七个与行业无关的类别：年报之后的最新定期报告与经营数据、最新股价与市值、卖方一致预期与目标价、可比公司及其估值倍数、行业规模与景气指标、竞争格局与份额、近三个月的重大事件。每个类别至少有一个已找到的问题，或在未找到时至少换过两个检索词。分析师读完资料包再写分析框架：3 ~ 4 条投资主线、驱动清单（年报位置 → 经营数据表 → 报告章节 → 预测参数）、外部调研清单和 3 ~ 5 个深度专题。深度专题从投资主线和资料中的分歧倒推，不预设行业套路；每个专题在阶段 4 写成一份带多个独立来源的笔记，在文稿中成为一节。

9.5 写作规范

主笔技能对“深度”提出了可检查的要求，下面几条是从人工质检中提炼的：

- **结构**。年报点评约占三分之一（业绩、盈利、资产负债与现金流、年报之后的最新进展），深度研究约占三分之二（行业空间、竞争对标、核心竞争力、深度专题、预期差、估值交叉验证）。
- **投资主线六项要求**。每条主线必须：落到可度量的驱动；说明与一致预期差在哪个变量、差多少；落到估值参数及其对每股价值的影响（引用敏感性、情景或反向 DCF 产物）；写出可观察的证伪条件；给出检验时点；回应最强的反方论据。补不齐的降为跟踪事项。公司判断与股价判断分开写。
- **预测依据**。关键预测参数必须写成“依据数字 → 换算 → 取值”的链条，排版工具包读取依据表时缺项直接报错。
- **跟踪表**。每条核心观点给出可逐期核对的指标及“确认 / 预警 / 证伪”三档数值阈值，阈值取自产物（基准与悲观情景假设、历史区间），因而同样过得了护栏。
- **措辞分层**。事实直接陈述（数字来自真值表）；推论用“主要系”“反映”；观点用“我们认为”“我们预计”。不使用“事实层面”（观点）”之类的行内标签。

9.6 复核编辑

复核编辑会话以“没有参与写作的研究所质控编辑”身份工作，读者设定为时间很少的买方基金经理。它先通读全文、不动笔，回答三个问题（核心结论是否一句话说得出、每条结论前是否有多环定量推导、故事线是否连贯），写在修改记录开头；再逐段问“去掉这一段读者会少知道什么”；然后做前后对账（同一指标只有一个数、结论与相邻图表一致、同一判断在摘要、正文与风险提示中方向一致）；最后整篇重读。技能说明列出了模型初稿中反复出现的机器腔写法——否定对照、自证与圆场、交代缺失、程序口吻、口语套话、数字罗列、无来源判断——但明确说明它们是判断线索而不是替换清单。需要补数字或改结论的问题写进人工复核清单 `review.md`，随交付物交给人。

9.7 排版

排版工具包 reportkit 提供读产物、标准图表与画图原语（含时间轴、流程图、文字表格等非数据图）、目录与图表目录、附录与免责声明、HTML 打印为 PDF 的能力。所有图表数据都从运行产物的 JSON 读取，不经过模型转抄。目录页码通过两遍打印回填；排版会话逐页查看渲染预览，修正留白、重叠与孤立标题。附录包含估值方法说明、关键假设汇总、全文数字的事实 / 推论 / 观点构成与统一免责声明。

10 评测体系与质检闭环

流水线的闸门保证交付物“齐全、新鲜、数字有出处”，但回答不了“这份研报好不好”。本节介绍我们的评分标准、计分方式、盲评设置、多轮质检闭环，以及如何校准评审模型本身。

10.1 评分标准的来源

评分标准由两条思路合成。**思路一**从“卖方深度报告在回答哪些问题”出发：按常规体例逐部分列出读者要的答案——首页要能复述“为什么值这个价”，核心观点要说明与市场的分歧在哪里，驱动拆解要有“外部驱动 → 量价结构 → 收入利润”的定量链条，风险要落到具体假设——每个问题改写成一条可二元判定的条目，并写清判定标准与反例。权重按对结论的影响分三档：决定估值的环节 5 分，支撑判断的关键论证 4 分，完

表 5　评分标准的层级结构（v2）。L0 不计分但失败即封顶；N 为负向条目，命中扣分。

层	条目	分值	判定者	内容
L0 硬闸门	5	—	程序	数字护栏、三表勾稽、DCF 与三表同口径、免责声明、评级与估值一致；任一失败总分封顶 40
L1 数据闭合	13	35	程序	抽取校验、分部预测基期闭合、股权价值桥加总、每股价值复算、情景排序、敏感性方向、终值占比、预测与历史衔接、正文估值与产物一致、外部信息可追溯、股价日期与来源、数字锁、人工复核清单
L2 分析质量	12	45	评审模型	核心结论、预期差、商业模式、驱动拆解、历史业绩归因、行业与竞争、预测假设依据、估值方法与参数、估值交叉验证、情景与敏感性、风险落到假设、跟踪指标与失效条件
L3 表达与合规	7	20	评审模型	故事线连贯、事实 / 推论 / 观点可区分、结论先行、无机器腔、数字密度、图表服务论点、假设与局限透明
N 负向	4	−30	程序 / 评审	无来源外部数据（−8，并封顶 70）、数字前后矛盾（−8，程序判）、聚合数构成描述错误（−6）、结论前后冲突（−8）

整性要求 2 ~ 3 分。思路二从工作流出发：把“年报 → 抽取 → 真值表 → 经营数据 → 预测 → 三表 → DCF → 文稿 → 排版”的每一次交接看作一个闭合点，上游的数在下游必须找得到、对得上、口径一致；这些条目几乎都能由程序判定，直接复用已有的校验工具。

10.2　计分

所有计分都在 Python 中完成，评审模型只输出逐条的 {id, 是否通过, 证据, 理由}。设 E_ℓ 为第 ℓ 层得分，$D = \max(\sum_{i \in N,\,\text{命中}} p_i,\ D_{\min})$ 为负向扣分（下限 $D_{\min} = -20$），$\mathscr{C}$ 为触发的封顶集合（L0 任一失败封顶 40；N-01 命中封顶 70），则

$$S = \min\Big(\max(0,\ E_{L1} + E_{L2} + E_{L3} + D),\ \min \mathscr{C}\Big). \tag{16}$$

缺判定的条目按未通过计（负向条目按未命中计）并单独列出。**交付标准**为：$S \geq 75$；L1、L2、L3 得分率分别不低于 80%、60%、60%；且未命中任何负向条目。权重、封顶与门槛都写在评分标准文件中，修改后可直接重算，无需重跑评审。

评分标准经历了三个版本。v1 依据三家 v0 研报的人工质检调整：外部信息可追溯（L1-10）从“来源文件是否存在”改为核对引用编号与外部数字的对应；人工复核清单（L1-13）只认填写完整的事项；“无机器腔”（L3-04）只看正文中成为习惯写法的程序口吻，图表来源与免责声明不算；“数字密度”（L3-05）把数字连同其同比、占比算作一个数据点。v2 把“数字前后矛盾”改为程序判定，把数值以外的结论冲突拆为单独的 N-04。

10.3　盲评

评审在仓库与工作区之外单独建立的目录中进行，目录里只有交付物：研报 PDF、文稿、估值产物（DCF、预测摘要、分部预测、情景、非经营性资产、可比公司）、外部来源登记、人工复核清单，以及评审条目（只含编号、标题、判定标准与反例，**不含分值与层级权重**）。评审会话看不到写作与复核过程、模型名称和其他公司的结果；只允许读本目录、写 `judgments.json`，不联网。评审说明要求：每一条都给出判定；完全满



图 **6**　质检闭环与信息隔离。生成侧（工作区内）看不到评分标准与分值；评测侧（仓库与工作区之外）只看到交付物。不达标时，问题单按路由回传：数据与假设类回到分析师 / 主笔，措辞与结构类交给复核编辑，版面类交给排版设计师。每轮存档，结束时回滚到得分最高的一轮。

足判定标准才算通过，**拿不准时判不通过**并写明缺什么；证据摘录原文并标注章节或页码。数据是否与年报一致、勾稽与估值算术是否正确已由程序核对，评审不必复算，只判断分析是否成立、论证是否充分、表达是否合格。

10.4　质检闭环

`refine` 把生成、复核与质检串成多轮循环（算法 3，图 6）。每条评分条目都带一个路由字段（`generator` 或 `reviewer`）和对应阶段；未通过的条目被写成问题单，只含问题描述、理由与证据，不含条目原文和分值。路由回生成侧的问题会删除初稿备份与修改记录，使复核从头再来，避免旧的复核结论覆盖新的文稿。

防止对评分标准过拟合。　多轮迭代天然有朝评分标准优化的风险（Goodhart 定律）。我们的对策有三：评分标准与完整评测结果放在工作区之外，生成方读不到；问题单只给理由与证据，不给条目原文、分值与通过线；L1 全部由程序判定，生成方无法通过措辞“说服”程序。设计中还预留了把评分标准拆成“反馈集”和“留出集”的方案：反馈集转成问题单，留出集只用于最终计分，两者得分差距过大即说明在应试。

10.5　校准评审模型

评审模型的判定只有在被检验之后才能用来驱动迭代。我们用三组证据回答三个不同的问题：

1. **稳定性：同一份研报重复盲评，判定是否一致？**对同一交付物重复盲评 k 次，计算逐条全一致的比例、两两不一致率与总分极差。
2. **灵敏度：已知缺陷能否判出？**对一份合格研报的文稿施加确定性的“劣化算子”，制造已知缺陷后重新排版、重新盲评（表 6）。缺陷是程序造的，“该判不通过”是已知真值。同时统计非目标条目被连带改判的数量，并与重复评审本身的波动对比。原稿已被判出该问题的变体不计入命中率。
3. **正确性：判出的问题，人是否认同？**人工质检页逐条复核评审判“不通过”的条目，计算判出问题的精确率。由于人工只复核了判不通过的条目，这组证据算不出漏判率，漏判由第 2 组补充。

算法 3 质检闭环（`refine`）

Require: 工作区 W，最大轮数 $K = 3$，最小提升 $\Delta = 2$

for $n = 1, \ldots, K$ **do**
  if $n = 1$ **then** 依次运行五个会话　▷ 已通过的阶段跳过
  else
    读取上一轮问题单 I，按路由分为 $I_{\text{gen}}, I_{\text{rev}}, I_{\text{layout}}$
    if $I_{\text{gen}} \neq \emptyset$ **then** 删除初稿备份；修订会话处理 I_{gen}（需要时重跑估值与护栏）；随后复核与排版按闸门续做
    else 复核编辑处理 I_{rev}（数字锁照常）；排版设计师处理 I_{layout}
    end if
  end if
  if `status`(W) 未全部通过 **then** 停止，交给人
  end if
  盲评并计分得 S_n，存档本轮文稿、复核文件与 PDF
  if 达标 **or** $S_n - S_{n-1} < \Delta \wedge S_{n-1} - S_{n-2} < \Delta$ **or** 无可回传问题 **then break**
  end if
end for
回滚到 $\arg\max_n S_n$（若期间重跑过估值则只标注最佳轮次）
若最佳轮仍未达标，把剩余问题追加进人工复核清单

表 6　劣化算子。每个算子只修改文稿，确定性地制造一种已知缺陷。

变体	制造的缺陷	目标条目
contradiction	只把摘要里的评级改成相反方向，正文不变	N-02
generic_risks	风险提示换成不对应任何假设的通用模板	L2-11
machine_tone	每两段注入一次程序口吻（如“根据程序计算结果”）	L3-04
no_cross_check	删掉相对估值与交叉验证小节	L2-09
no_expectation_gap	删掉市场关注问题、预期差与“股价隐含”专题	L2-02
no_tracking	删掉跟踪指标与结论失效条件	L2-12
unsourced_number	行业部分插入一段无来源的市场规模与份额数字	N-01

11　实验

本节报告五组结果：自造样例上的端到端指标、真实年报上的抽取覆盖、引擎一致性、研报质检得分的演进，以及评审模型的校准。我们先说明一个总的前提：这些结果来自一支学生团队在一个月开发周期内的内部评测，样本量小，版本之间不是严格的对照实验；我们在每组结果后写明它能说明什么、不能说明什么。

11.1　实验设置

模型。 研报流水线中，研究助理、分析师、主笔与排版设计师使用 Claude Opus 5.5，复核编辑与盲评使用 Claude Sonnet 5.5，均通过 Claude Code 的无头模式（`claude -p`，事件流输出）运行。计算引擎不调用任何模型。

数据。 自造样例为一家虚构制造业公司“星河智造”的 2024 年年报、三季报、估值假设、可比公司、一致预期、业绩预告与一份植入了 7 处错误的研报草稿，内部完全自洽。真实数据为贵州茅台、宁德时代、招商轮船、中际旭创的 2025 年年报，以及宇树科技的招股说明书；原件及其 SHA-256 登记在仓库中。

11.2 自造样例上的端到端指标

表 7 自造样例上的评测结果（`eval/run_eval.py`）。样例为虚构且内部自洽，指标只说明流程正确，不代表真实效果。

指标	结果
字段抽取准确率	100%（35/35）
纠错召回率 / 精确率	100% / 100%（7 处植入错误）
误报数	0
错误类型判定准确率	100%（口径 1、数值 4、单位 1、期间 1）

表 7 的满分不应被解读为系统能力：样例是我们自己构造的，与代码共同演进。它的作用是回归测试——任何改动使这些指标下降都被视为回归。

11.3 真实年报上的抽取

我们在贵州茅台、宁德时代、招商轮船三家 2025 年年报上，由分析师会话抽取并经 `verify-facts` 校验、人工抽查后得到的 314 个数值作为核验集，检验规则通道（不调用任何模型）能独立取到多少、取对多少。

表 8 规则通道在三家真实年报上的覆盖（合计 314 个核验值）。改进指第 6.2 节的四项版式处理。

版本	取到的核验值	覆盖率	与核验值错配
改进前	128	40.8%	—
改进后（稀疏网格、跨页续表、科目折行、同页多单位）	293	93.3%	0

覆盖率从 40.8% 提升到 93.3%，且取到的 293 个值与核验值零错配。剩余未取到的主要是股本变动表、政府补助口径与部分季度扣非别名。这一结果的意义在于双通道校验：当规则通道能独立覆盖九成以上的核心数值时，模型通道的任何一个读错都会以冲突的形式暴露出来，而不会静默进入真值表。局限是只有三家公司、核验集由我们自己的流程产生；扩充到 15 家左右、覆盖更多行业与版式的评测集正在收集中。

11.4 计算引擎的一致性与完整性

截至 2026 年 10 月 5 日，开发分支上共 166 个测试，161 个通过，5 个因本地没有原参照插件而跳过（逐项对照测试），覆盖流水线、季报拆分、三表、DCF、反向 DCF、研报排版、护栏与出处分类、一致性检查、质检与闸门。三表、情景表、业绩对比与 DCF 引擎在样例上与原参照引擎的快照完全相等。在真实研报的运行中，三表模型的六项勾稽全部通过，DCF 各期收入、EBIT、折旧摊销、资本开支与营运资本与三表模型的最大偏差为 7.3×10^{-12} 百万元（第 11.7 节）。

11.5 研报质检得分的演进

我们用同一份评分标准（v0）、同一评审模型对三个流水线版本的交付稿各评一次：

- **旧流程（v0）**：贵州茅台、宁德时代、招商轮船。没有前置的资料搜集阶段，也没有独立的复核会话与数字锁。
- **先搜集（v1）**：中际旭创、宇树科技。研究助理会话先联网搜集并登记来源，独立复核会话改写并过数字锁。

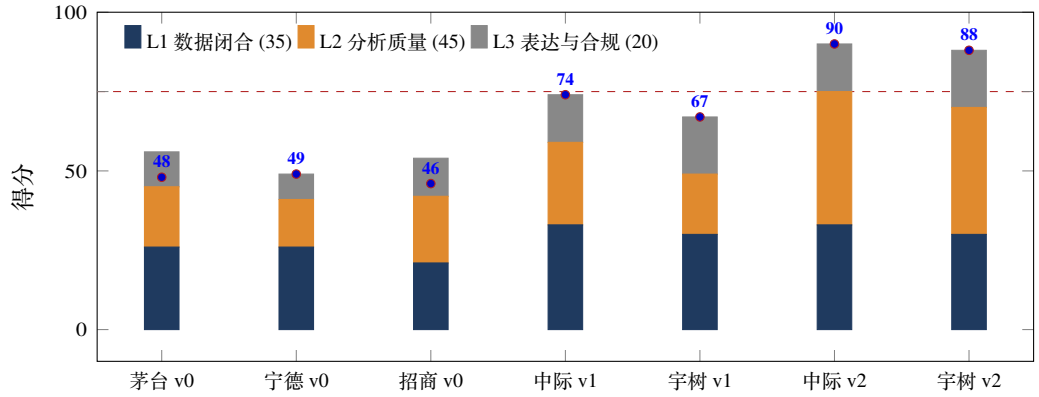


图 7　三个流水线版本交付稿的质检得分（评分标准 v0，单次盲评）。柱为各层正向得分，红点为扣分后的总分（茅台、招商各命中一次负向条目“前后矛盾”，−8），红色虚线为交付线 75 分。v0 与 v1/v2 的公司不同，v1 与 v2 为同一组公司。

表 9　质检得分明细（评分标准 v0，单次盲评）。

版本	公司	L1 /35	L2 /45	L3 /20	扣分	总分	达标
v0 旧流程	贵州茅台	26	19	11	−8	48	否
	宁德时代	26	15	8	0	49	否
	招商轮船	21	21	12	−8	46	否
v1 先搜集	中际旭创	33	26	15	0	74	否
	宇树科技	30	19	18	0	67	否
v2 深度调研	中际旭创	33	42	15	0	90	是
	宇树科技	30	40	18	0	88	是

- **深度调研（v2）**：同为中际旭创、宇树科技。在 v1 基础上增加分析框架中的深度专题与专题调研阶段，以及篇幅与图表闸门。

值得注意的是，中际旭创的 v1 与 v2 稿使用了同一套估值结果（基准每股价值均为 521.72 元，L1 各项证据相同），两版的差别集中在调研深度与文稿：登记的检索问题由 21 个增至 31 个，来源由 33 个增至 51 个，正文来源引用由 67 处增至 243 处。

观察。 图 7 与表 9 显示两段提升。从 v0 到 v1，L1 数据闭合的提升最直接：v0 三家在“外部信息可追溯”（L1-10）、“复核改写过数字锁”（L1-12）、“人工复核清单”（L1-13）上全部失分，这三项分别由资料搜集先行与来源编号、独立复核会话和复核清单补上。从 v1 到 v2，同一组公司的 L1、L3 几乎不变，提升几乎全部来自 L2 分析质量（中际旭创 26 → 42，宇树科技 19 → 40），与 v2 新增深度专题调研的设计意图一致。v0 三家在 L2 中的“行业与竞争”“预测假设有依据”“估值方法与参数”“估值交叉验证”四条全部失分，而这些在 v2 中大多通过。

这组结果不能说明什么。 第一，每份交付稿只评了一次，而第 11.6 节显示重复评审的总分极差为 7 ~ 8 分，单次得分的差异需要结合这一波动来读。第二，v0 与 v1/v2 的公司不同，行业难度不可比。第三，版本之间同时改变了多个因素（资料搜集、复核会话、深度专题、篇幅闸门），这里的比较不是消融实验，不能把提升归因到某一项设计上。第四，评审与生成模型同属一个模型家族，可能存在自我偏好 [14]。

11.6 评审模型的校准

稳定性。 我们对中际旭创与宇树科技的 v2 交付稿各重复盲评 3 次（表 10）。逐条全一致的比例为 91% 与 86%，两两不一致率为 0.061 与 0.091，总分极差为 7 与 8 分。不稳定的条目集中在“预测假设有依据”（L2-07）与“假设与局限透明”（L3-07），二者的判定都依赖“关键假设”“基本满足”这类需要裁量的措辞。

表 10 同一交付稿重复盲评 3 次的稳定性（评审判定的 22 个条目）。

公司	逐条全一致	两两不一致率	三次总分	极差	不稳定条目
中际旭创	20/22（91%）	0.061	85 / 80 / 78	7	L2-07、L3-07
宇树科技	19/22（86%）	0.091	80 / 88 / 83	8	L2-07、L3-07、N-02

灵敏度。 表 11 给出七个劣化算子在两家公司上的结果。三类缺陷能稳定判出：插入无来源数字（2/2，同时触发数字护栏，总分被封顶为 40）、注入程序口吻（2/2）、摘要评级反转（2/2，但原稿在重复评审中已被判出过“前后矛盾”，按规则不计入命中率）。整节删除类缺陷——删交叉验证、删预期差、删跟踪指标、风险泛化——大多未判出，按有效变体计，中际旭创判出 2/6，宇树科技判出 3/6。

表 11 劣化命中。“ ”表示劣化稿上目标条目被判出问题。连带改判为非目标条目与原稿多数判定不同的数量。

	中际旭创		宇树科技	
变体（目标条目）	判出	连带改判	判出	连带改判
unsourced_number（N-01）		3		2
machine_tone（L3-04）		2		3
contradiction（N-02）†		2		2
generic_risks（L2-11）		1		2
no_cross_check（L2-09）		1		2
no_expectation_gap（L2-02）		2		4
no_tracking（L2-12）		2		3
有效变体判出率	2/6（33%）		3/6（50%）	

† 原稿已被判出该问题，不计入判出率。

我们逐条阅读了未判出变体的评审理由，发现多数情况不是评审漏判，而是**劣化不彻底**：深度研报会在核心观点、深度专题和图表中多次重复同一论点，而图表由排版脚本从产物生成、不在文稿里，删除文稿中的一节删不干净。例如删除“预期差”一节后，评审仍在摘要与估值章中找到了“股价隐含约 18 倍退出 EV/EBITDA”这一分歧表述；删除跟踪一节后，评审指出该节在 PDF 中“只有标题没有内容”，但失效条件散见于预期差与专题结论中，因此按判定标准仍判通过。要测量真正的漏判率，劣化算子需要改成跨全文删除相关句子并同步删除图表，这是下一步的工作。这组结果同时提示了评分标准本身的一个问题：对“是否存在”的判定容易被零散内容满足，可能需要区分“有专门一节”与“有零散提及”。

与人工的一致性。 两位复核人对三家 v0 研报中评审判“不通过”的 33 条进行了复核：确认 26 条、误报 5 条、待查 2 条，判出问题的精确率为 84%（不含待查）。误报集中在两

类:“无机器腔”判得过严（两家研报因少量“不是……而是……”句式被判不通过）与“数字密度”的口径（数字加同比是否算堆砌）。评分标准 v1 据此修改了这两条的判定标准（第 10 节）。

11.7 案例：一份 v2 研报的程序检查

以中际旭创 v2 交付稿为例，表 12 摘录 L0 与 L1 中程序检查给出的具体证据。这些数字均来自评测产物（`eval/results/v2_ 中际旭创/`），对应的估值结论高度依赖文中假设，不构成投资建议。

表 12 中际旭创 v2 研报的部分程序检查结果。

检查	程序给出的证据	结果
L0-01 数字护栏	无出处数字 0，指标期间不符 0，外部数字缺引用 0	
L0-02 三表勾稽	6 项勾稽，失败 0	
L0-03 DCF 与三表同口径	收入、EBIT、折旧摊销、资本开支、营运资本逐期核对，最大偏差 7.3×10^{-12} 百万元	
L0-05 评级与估值一致	基准每股价值 521.72 元，参考股价 808.44 元，上行空间 −35.5%，按规则为“卖出”，正文一致	
L1-02 分部预测基期闭合	分部合计 38,239.94 百万元，等于营业收入，差 0.00	
L1-05 情景排序	悲观 185.77 / 基准 521.72 / 乐观 918.06 元	
L1-07 终值占比	终值占企业价值 77.1%（阈值 80%）	
L1-08 预测与历史衔接	首个预测年收入增速较实际年变动 71.0 个百分点（阈值 30）	
L1-10 外部信息可追溯	31 个问题、61 次检索、51 个来源，正文引用 243 处	
L1-12 数字锁	复核改写删去 15 个数字，无新增	
L1-13 人工复核清单	`review.md` 10 条	

L1-08 的失败值得一提：程序发现首个预测年的收入增速相对实际年变动过大。这不一定是错误——高景气行业的增速确实可能大幅变化——但它是一个需要人看的信号，因此被计为失分并进入问题单。评审在 L2 中也给出了具体的批评：估值方法的选择理由与 β、风险溢价、永续增长率的依据不足（L2-08 未通过），同业倍数与 DCF 的口径不一致。

11.8 成本与耗时

一次盲评约需 3 ~ 4 分钟：中际旭创与宇树科技 v2 稿的评审分别耗时 163 秒与 254 秒，约 17 轮工具调用，输入约 96 万 ~ 107 万词元，按 API 价格折合约 2.3 美元。生成一份研报（五个会话、11 个阶段）的墙钟时间约 30 ~ 60 分钟，主要花在资料搜集、深度调研与排版上。

12 局限、风险与合规

评测规模。 真实年报抽取只覆盖三家公司，研报质检只覆盖五家公司、七份交付稿，每份只评一次。自造样例上的满分指标只能说明流程正确。我们正在收集约 15 家、覆盖消费、制造、周期、科技、公用事业等行业与沪深北三地版式的年报与季报，以及针对其中 8 ~ 10 家的卖方研报人工埋错集（计划 150 ~ 200 处，其中约 20% 为“正确但容易误报”的断言），用于更可靠地评估抽取与纠错。

非对照比较。 研报得分的演进同时改变了多个因素，不是消融实验。我们计划在扩充评测集后，分别去掉资料搜集先行、复核会话与数字锁、深度专题三项设计，测量各自的贡献。

评审模型的局限。 评审与生成模型属于同一模型家族，可能偏好同族生成的文本。重复评审的总分极差为 7 ~ 8 分，接近交付线附近的得分差。整节删除类缺陷的判出率偏低，虽然主要原因是劣化不彻底，但也暴露了"是否存在"类条目容易被零散内容满足的问题。人工复核只覆盖评审判"不通过"的条目，无法估计漏判率。

闸门是数量下限。 字数、页数、图表数、来源数等闸门只能阻止模型偷懒，不能认证质量；一份满足全部数量闸门的研报仍可能论证薄弱，这部分只能靠盲评与人工复核兜底。

护栏的边界。 护栏核对的是"有出处"与"口径明确的指标绑定"，不核对预测是否合理、外部来源是否权威、分析判断是否正确。预测数字之所以能通过护栏，是因为它们出现在计算产物中，而这些产物本身由模型起草的假设驱动；假设的合理性要靠假设表中的依据、反向 DCF 的对照以及人工复核清单来约束。

合规与投资建议。 所有估值参数均为起草的假设，参考股价为外部数据，估值结论高度依赖这些假设。系统生成的评级由固定规则从估值结果得出，研报统一附"不构成投资建议"的免责声明，不提供加仓、减仓等操作建议。卖方研报与年报原件的版权归原作者所有：评测集中的卖方研报只用于内部评测，仓库只保存我们抽出的断言与标签。第三方代码的参照关系、版本与许可证在 `THIRD_PARTY.md` 中逐项列明。

13 结论与后续计划

本文介绍了 FinAgent，一个从年报到深度研报的可追溯投研多智能体系统。它的出发点很朴素：模型擅长读、写与判断，不擅长、也不应该负责计算和记忆数字。围绕这一分工，我们把九条设计规范写成程序可检查的不变量，构建了证据约束的双通道抽取、单一真值表驱动的估值链、按展示精度定容差的数字护栏与约束改写的数字锁；在此之上，用隔离工作区、五个上下文独立的会话与十一道闸门组织研报生产，用对生成方不可见的四层评分标准与盲评闭环组织质检，并用重复评审、程序造缺陷与人工复核三组证据校准评审模型本身。

初步结果表明：面向真实年报版式的解析让规则通道在三家公司上覆盖了 93% 的核验值且零错配，使双通道校验在真实数据上真正起作用；先搜集资料、再做深度专题调研的流水线显著提升了研报的分析质量得分；评审模型在可由文本局部判断的缺陷上可靠，在需要通读全文判断"是否存在"的条目上仍需改进评分标准与校准方法。

决赛前的计划按优先级排列：（1）扩充真实年报与人工埋错研报评测集，报告抽取与纠错在多行业上的指标；（2）改进劣化算子（跨全文删除并同步删除图表）并完成三项设计的消融；（3）季报的规则通道表头识别（"本报告期 / 年初至报告期末"），让交叉校验覆盖季报；（4）完善终端界面与运行回放，用于现场演示时从研报中的任一数字回溯到年报页码。

参考文献

[1] Rahul K. Arora et al. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.

[2] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

[3] Zhiyu Chen et al. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[4] Zhiyu Chen et al. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

[5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.

[6] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[7] Sirui Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Pranab Islam et al. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

[9] Ziwei Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

[10] Seungone Kim et al. Prometheus: Inducing fine-grained evaluation capability in language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[13] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[15] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[16] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[17] Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[18] Shijie Wu et al. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

[19] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. FinGPT: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023.

[20] Hongyang Yang et al. FinRobot: An open-source AI agent platform for financial applications using large language models. *arXiv preprint arXiv:2405.14767*, 2024.

[21] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[22] Yangyang Yu et al. FinMem: A performance-enhanced LLM trading agent with layered memory and character design. *arXiv preprint arXiv:2311.13743*, 2023.

[23] Wentao Zhang et al. A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.

[24] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[25] Fengbin Zhu et al. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

附录 A　复现说明

环境：Python 3.12；计算引擎只依赖标准库与 pdfplumber。深度研报流水线另需 Node.js 18 以上、已登录的 Claude Code、Chrome / Chromium / Edge（HTML 打印为 PDF）与中文字体。`scripts/setup.sh`（Windows 为 `setup.ps1`）创建虚拟环境、安装依赖并检查上述组件，最后运行 `python -m finagent.doctor` 逐项自检。

自造样例端到端（不调用模型）。

```
python -m finagent all --input data/samples/annual_report_2024.md --year 2024 \
  --company 星河智造 --assumptions data/samples/valuation_assumptions.json \
  --report data/samples/research_draft.md --peers data/samples/peers.csv \
  --estimates data/samples/estimates.csv --guidance data/samples/guidance.csv --llm off
python eval/run_eval.py          # 样例评测指标
python -m pytest -q tests        # 测试与一致性快照
```

真实年报的计算复算（不调用模型）。 抽取文件随仓库保存在 `data/extractions/<公司>_<年度>/`，可直接用其运行校验与估值（`all` 另需研报草稿 `--report`，没有草稿时用 `valuation`；以贵州茅台为例，两条命令各约 5 秒，三表勾稽全部通过）：

```
python -m finagent verify-facts --input <年报.pdf> --facts data/extractions/<公司>_2025/facts.jso
python -m finagent valuation --input <年报.pdf> --year 2025 --company <公司> \
  --facts data/extractions/<公司>_2025/facts.json \
  --extras data/extractions/<公司>_2025/extras.json \
  --assumptions data/extractions/<公司>_2025/valuation_assumptions.json --llm off
```

研报流水线与质检（需要模型环境）。

```
python -m finagent.report_pipeline build --company <公司> --year 2025 \
  --price <股价> --price-source "<来源>"
python -m finagent.report_pipeline refine --workspace ~/finagent-samples/<公司>_2025
python -m finagent.report_pipeline status --workspace ~/finagent-samples/<公司>_2025
python -m finagent.report_pipeline collect --workspace ~/finagent-samples/<公司>_2025
```

评分重算与校准。 修改评分标准后无需重跑评审即可重算；校准报告由已保存的判定汇总生成：

```
python -m finagent.evaluation score --result eval/results/v2_中际旭创
python -m finagent.evaluation.calibrate report
```

附录 B　评分标准条目

表 13 列出 L1 ~ L3 的全部条目与分值（v2）。L0 与负向条目见表 5。完整判定标准与反例见 `eval/rubrics/report_v2.json`。

附录 C　会话提示要点

各会话的启动提示都很短，只规定身份、阶段边界和收尾动作；做法写在技能说明与工作区说明里。以下为要点（原文见 `finagent/report_pipeline.py`）。

表 13　L1 ~ L3 条目与分值。

编号	条目	分	编号	条目	分
L1-01	抽取校验	3	L2-01	核心结论	4
L1-02	分部预测基期闭合	3	L2-02	预期差	4
L1-03	股权价值桥加总	4	L2-03	商业模式	3
L1-04	每股价值复算	3	L2-04	驱动拆解	5
L1-05	情景排序	2	L2-05	历史业绩归因	4
L1-06	敏感性方向	2	L2-06	行业与竞争	4
L1-07	终值占比（≤ 80%）	2	L2-07	预测假设有依据	5
L1-08	预测与历史衔接	2	L2-08	估值方法与参数	3
L1-09	正文估值与产物一致	3	L2-09	估值交叉验证	4
L1-10	外部信息可追溯	4	L2-10	情景与敏感性	3
L1-11	股价有日期与来源	2	L2-11	风险落到假设	4
L1-12	复核改写过数字锁	3	L2-12	跟踪指标与失效条件	2
L1-13	人工复核清单	2			
L3-01	故事线连贯	4	L3-05	数字密度	2
L3-02	事实、推论、观点可区分	3	L3-06	图表服务论点	3
L3-03	结论先行	3	L3-07	假设与局限透明	2
L3-04	无机器腔	3			

- **研究助理**：只做阶段 1；先读资料搜集方法，用联网检索把年报之后的最新信息搜齐，登记来源并写资料包；不做财务抽取、不写分析框架；结束时运行 `status`，阶段 1 通过即停止。
- **分析师**：先读资料包，再完成阶段 2 ~ 7（抽取、框架与深度专题、深度调研、经营数据、估值假设、估值运行）；不写文稿；每个阶段结束运行 `status`。
- **主笔**：先读框架、资料包、专题笔记与运行产物，写“年报点评 + 深度研究”，深度研究要做多环定量推导；护栏与篇幅闸门通过即停止，复核与排版由后续会话完成。
- **复核编辑**：没有参与写作；读复核技能说明，只做阶段 9。
- **排版设计师**：文稿已定稿、不改正文；从模板起写本公司排版脚本，每张图服务一个论点，逐页检查预览。
- **修订（质检回传）**：读问题单，修改对应阶段产物；需要补数据、改假设就重跑估值，文稿改完重跑护栏；处理不了的写进复核清单并说明原因。

所有提示都要求命令直接以工作区内的解释器包装 `./py` 开头、不加 `cd`、不串联，以匹配工具白名单。

附录 D　图的生成

图 1 ~ 6 的示意图由图像生成模型按统一风格绘制，提示词与风格约束保存在 `paper/figures/PROMPTS.md`；图 7 与全部表格由仓库中的评测结果直接生成，未经图像模型处理。示意图只表达结构，不含任何实验数字。